Beginner's Guide To TypeScript + React Integration

From TypeScript Basics to React Integration

Created by Yogesh Chavan

yogeshchavan.dev

https://www.linkedin.com/in/yogesh-chavan97/

Table of Contents

Section 5: Advanced Types

Section 6: Generics

Section 7: TypeScript with React

1. Introduction to TypeScript

What is TypeScript?

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale. Developed and maintained by Microsoft, TypeScript adds optional static typing and class-based object-oriented programming to JavaScript. Since TypeScript is a superset of JavaScript, existing JavaScript programs are also valid TypeScript programs.

TypeScript code cannot run directly in browsers or Node.js - it must first be compiled (transpiled) to JavaScript. This compilation step is where TypeScript catches type errors, helping you find bugs before your code runs.

Why TypeScript over JavaScript?

While JavaScript is powerful and flexible, it has limitations when building large-scale applications. TypeScript addresses these limitations:

- **Static Type Checking** - Catch errors at compile time rather than runtime
- **Better IDE Support** - Enhanced autocompletion, navigation, and refactoring
- **Improved Code Readability** - Type annotations serve as documentation
- **Enhanced Refactoring** - Safely rename symbols and restructure code
- **Latest ECMAScript Features** - Use modern features with backward compatibility

TypeScript vs JavaScript

Feature	JavaScript	TypeScript
Type System	Dynamic typing	Static typing (optional)
Compilation	Interpreted directly	Compiles to JavaScript
Error Detection	Runtime errors	Compile-time errors
IDE Support	Basic	Enhanced (IntelliSense)
Interfaces	Not supported	Fully supported
Generics	Not supported	Fully supported

Installing TypeScript

Step 1: Install Node.js

Before using TypeScript, you need Node.js installed on your machine. Follow these steps:

1. Visit the official Node.js website: https://nodejs.org
2. Download the LTS (Long Term Support) version for your operating system
3. Run the installer and follow the installation wizard
4. Verify the installation by opening a terminal and running:

```
node --version
# Should output: v20.x.x or higher

npm --version
# Should output: 10.x.x or higher
```

Step 2: Install TypeScript Globally

Once Node.js is installed, you can install TypeScript globally using npm:

```
npm install -g typescript
```

Step 3: Verify TypeScript Installation

```
tsc --version
# Should output: Version 5.x.x
```

Tip: You can also install TypeScript locally in a project using npm install typescript --save-dev. This is recommended for projects to ensure consistent versions across team members.

Setting up TypeScript with Node.js

To set up a new TypeScript project, create a directory and initialize it:

```
mkdir my-typescript-project
cd my-typescript-project
npm init -y
npm install typescript --save-dev
```

Create a TypeScript configuration file (tsconfig.json):

```
npx tsc --init
```

Here's a recommended configuration for beginners:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "./dist",
    "rootDir": "./src"
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

Your First TypeScript Program

Create a file called hello.ts in your src folder:

```
// src/hello.ts
function greet(name: string): string {
    return "Hello, " + name + "!";
}

const message: string = greet("TypeScript");
console.log(message);
```

Notice the type annotations: `name: string` specifies the parameter type, and `: string` after the parentheses specifies the return type.

Understanding the Compilation Process

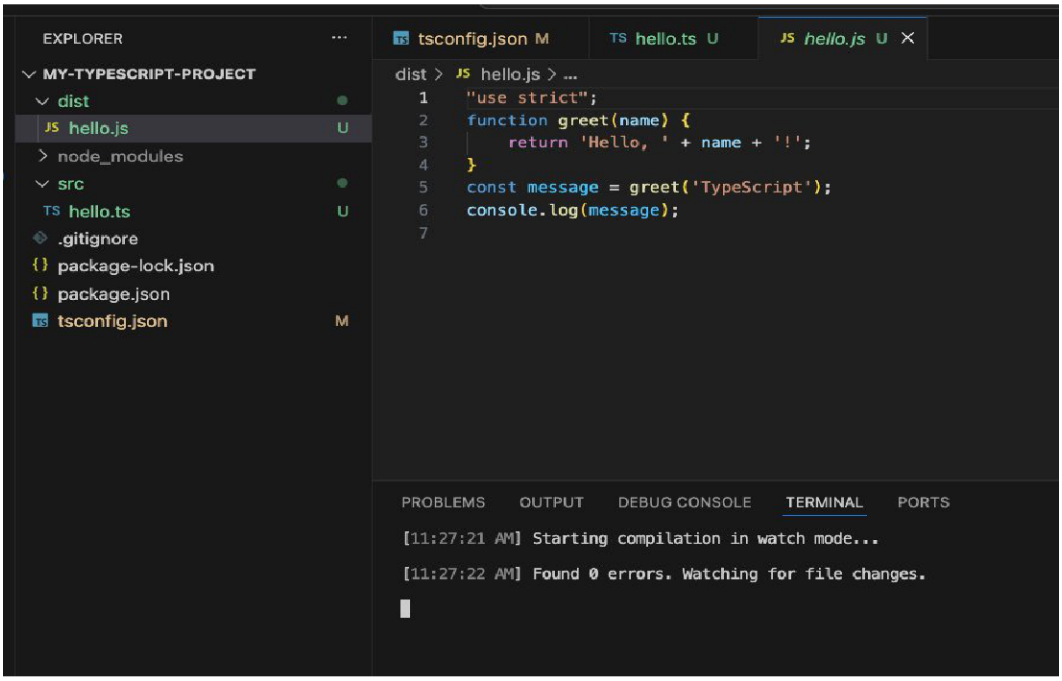
TypeScript must be compiled to JavaScript before it can run:

```
# Compile a single file
tsc src/hello.ts

# Compile using tsconfig.json
tsc

# Watch mode - recompile on changes
tsc --watch
```

When you run the `tsc` command (or `tsc --watch` for continuous compilation), TypeScript creates a new `dist` folder containing the compiled JavaScript files. You'll see your `hello.ts` file transformed into `hello.js` with all type annotations removed:

```
// dist/hello.js (compiled output)
"use strict";
function greet(name) {
    return 'Hello, ' + name + '!';
}
const message = greet('TypeScript');
console.log(message);
```

Here's what the project structure looks like in VS Code after compilation:



Notice in the terminal that the TypeScript compiler found 0 errors and is watching for file changes. The `dist` folder now contains the compiled `hello.js` file.

> **Note:** All type annotations are removed during compilation. Types only exist at development and compile time - they help catch errors but don't affect runtime behavior.

Running the Compiled Code

Now that your TypeScript code has been compiled to JavaScript, you can run it using Node.js. Execute the compiled JavaScript file from the `dist` folder:

```
# Run the compiled JavaScript file
node dist/hello.js
```

You should see the following output in your terminal:

```
Hello, TypeScript!
```

Congratulations! You've successfully written, compiled, and executed your first TypeScript program. The workflow is: write `.ts` files → compile with `tsc` → run the output `.js` files with `node`.

Tip: You can also use ts-node to run TypeScript files directly without manual compilation: npm install -g ts-node, then run ts-node src/hello.ts. This is useful during development.

2. TypeScript Basics

Basic Types (string, number, boolean)

TypeScript provides several basic types that form the foundation of the type system:

```
// String type
let firstName: string = "John";
let lastName: string = 'Doe';

// Number type (includes integers and floats)
let age: number = 30;
let price: number = 99.99;
let hex: number = 0xf00d;

// Boolean type
let isActive: boolean = true;
let isCompleted: boolean = false;
```

Type Inference

TypeScript can automatically infer types based on assigned values:

```
// TypeScript infers the type automatically
let name = "Alice";        // inferred as string
let count = 42;            // inferred as number
let isValid = true;        // inferred as boolean

// Type is locked after inference
name = "Bob";              // OK
name = 123;                // Error: Type 'number' is not
                           // assignable to type 'string'
```

Best Practice: Use explicit types when the initial value doesn't clearly indicate the intended type, or when declaring variables without immediate initialization.

Arrays and Tuples

TypeScript provides two ways to define arrays, and introduces tuples for fixed-length arrays:

```
// Arrays - two syntax options
let numbers: number[] = [1, 2, 3, 4, 5];
let names: Array<string> = ["Alice", "Bob", "Charlie"];

// Mixed arrays
let mixed: (string | number)[] = [1, "two", 3];

// Tuples - fixed length with specific types
let person: [string, number] = ["Alice", 30];
let coordinate: [number, number, number] = [10, 20, 30];

// Accessing tuple elements
console.log(person[0]);    // "Alice" (string)
console.log(person[1]);    // 30 (number)
```

Enums

Enums define a set of named constants, making code more readable:

```typescript
// Numeric enum (auto-increments from 0)
enum Direction {
    Up,      // 0
    Down,    // 1
    Left,    // 2
    Right    // 3
}

// Numeric enum with custom values
enum StatusCode {
    OK = 200,
    BadRequest = 400,
    NotFound = 404,
    ServerError = 500
}

// String enum
enum Color {
    Red = "RED",
    Green = "GREEN",
    Blue = "BLUE"
}

// Using enums
let direction: Direction = Direction.Up;
let status: StatusCode = StatusCode.OK;
```

Using 'as const' Instead of Enums

While enums are useful, modern TypeScript development often favors using `as const` assertions instead. The `as const` assertion tells TypeScript to infer the most specific type possible, making values readonly and literal types.

Why use 'as const' over enums?

- **Tree-shaking friendly** - Regular objects with 'as const' can be tree-shaken by bundlers, while enums cannot
- **No runtime overhead** - 'as const' objects don't generate extra JavaScript code like enums do
- **Better type inference** - Works seamlessly with TypeScript's type system
- **Simpler JavaScript output** - The compiled code is just a plain object

```typescript
// Using 'as const' instead of enums
const Direction = {
    Up: "UP",
    Down: "DOWN",
    Left: "LEFT",
    Right: "RIGHT"
} as const;

// Create a type from the object values
type Direction = typeof Direction[keyof typeof Direction];
// Type is: "UP" | "DOWN" | "LEFT" | "RIGHT"

// Usage
let move: Direction = Direction.Up;  // OK
move = "UP";                          // Also OK
move = "DIAGONAL";                    // Error!

// Another example with status codes
const StatusCode = {
    OK: 200,
    BadRequest: 400,
    NotFound: 404,
    ServerError: 500
} as const;

type StatusCode = typeof StatusCode[keyof typeof StatusCode];
// Type is: 200 | 400 | 404 | 500
```

Best Practice: For new projects, prefer 'as const' objects over enums. They provide the same benefits with better bundle size and simpler JavaScript output. Use enums only when you need reverse mapping (numeric enums) or when working with legacy codebases.

any, unknown, and never

TypeScript provides special types for handling dynamic values:

```typescript
// any - opts out of type checking (avoid when possible)
let flexible: any = 4;
flexible = "string";      // OK
flexible = true;          // OK
flexible.anything();      // OK (but risky!)

// unknown - type-safe alternative to any
let uncertain: unknown = 4;
uncertain = "string";     // OK

// Must check type before using
if (typeof uncertain === "string") {
    console.log(uncertain.toUpperCase()); // OK
}

// never - represents values that never occur
function throwError(message: string): never {
    throw new Error(message);
}

function infiniteLoop(): never {
    while (true) {}
}
```

Tip: Prefer 'unknown' over 'any' when you don't know the type. It forces type checking before use, making your code safer.

void Type

The void type represents the absence of a return value:

```typescript
// Function that doesn't return anything
function logMessage(message: string): void {
    console.log(message);
}

// Arrow function with void return
const printNumber = (num: number): void => {
    console.log(num);
};
```

Type Assertions

Type assertions tell TypeScript you know more about a value's type:

```typescript
// Two syntax options
let someValue: unknown = "Hello, TypeScript!";

// Angle-bracket syntax
let strLength1: number = (<string>someValue).length;

// "as" syntax (required in React - JSX)
let strLength2: number = (someValue as string).length;

// Working with DOM elements
const input = document.getElementById("myInput") as HTMLInputElement;
input.value = "Hello!";

// Non-null assertion
function getValue(arr: number[], index: number): number {
    return arr[index]!; // Assert it won't be undefined
}
```

The `!` operator is called the **non-null assertion operator**. It tells TypeScript that you are certain the value will not be `null` or `undefined`, even though TypeScript thinks it might be. In the example above, `arr[index]` could potentially be `undefined` if the index is out of bounds, but using `!` tells TypeScript "trust me, this value exists."

```typescript
// More examples of non-null assertion (!)
const button = document.getElementById("submit")!;
// Without !, TypeScript thinks button could be null

// Use when you're certain a value exists
interface User {
    name: string;
    email?: string;  // optional property
}

function sendEmail(user: User) {
    // We checked elsewhere that email exists
    const email = user.email!;  // Assert it's not undefined
    console.log("Sending to:", email);
}
```

Warning: Use the non-null assertion (!) sparingly. It bypasses TypeScript's null checks, so incorrect usage can lead to runtime errors. Prefer proper null checks (if statements or optional chaining ?.) when possible.

Literal Types

Literal types specify exact values a variable can hold:

```
// String literal types
type Direction = "north" | "south" | "east" | "west";
let heading: Direction = "north";  // OK
heading = "northeast";             // Error!

// Numeric literal types
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;
let roll: DiceRoll = 4;            // OK
roll = 7;                          // Error!

// In function parameters
function move(direction: "up" | "down" | "left" | "right") {
    console.log(`Moving ${direction}`);
}

move("up");       // OK
move("diagonal"); // Error!
```

3. Functions in TypeScript

Function Type Annotations

TypeScript allows type annotations on function parameters and return values:

```typescript
// Basic function with types
function add(a: number, b: number): number {
    return a + b;
}

// Function type as a variable
let multiply: (x: number, y: number) => number;
multiply = function(x, y) {
    return x * y;
};

// Type alias for function types
type MathOperation = (a: number, b: number) => number;

const subtract: MathOperation = (a, b) => a - b;
const divide: MathOperation = (a, b) => a / b;

// Function with object parameter
function printUser(user: { name: string; age: number }): void {
    console.log(`${user.name} is ${user.age} years old`);
}
```

What Happens When You Pass Wrong Types?

TypeScript catches type mismatches at compile time, preventing runtime errors before your code even runs:

```
// Calling functions with wrong types
add(5, 10);           // OK: returns 15
add("5", 10);         // Error: Argument of type 'string' is not
                      // assignable to parameter of type 'number'

multiply(3, 4);       // OK: returns 12
multiply(3, "4");     // Error: Argument of type 'string' is not
                      // assignable to parameter of type 'number'

subtract(10, 5);      // OK: returns 5
subtract(10);         // Error: Expected 2 arguments, but got 1

printUser({ name: "Alice", age: 30 });  // OK
printUser({ name: "Bob" });             // Error: Property 'age' is
                                        // missing in type '{ name: string; }'
printUser("Alice");                     // Error: Argument of type 'string'
                                        // is not assignable to parameter
```

Key Benefit: These errors appear in your IDE as you type and during compilation - not at runtime. This is one of TypeScript's biggest advantages: catching bugs before your code runs!

Optional & Default Parameters

Functions can have optional (?) and default parameter values:

```typescript
// Optional parameters (must come after required)
function greet(name: string, greeting?: string): string {
    if (greeting) {
        return `${greeting}, ${name}!`;
    }
    return `Hello, ${name}!`;
}

greet("Alice");           // "Hello, Alice!"
greet("Bob", "Hi");       // "Hi, Bob!"

// Default parameters
function createUser(
    name: string,
    role: string = "user",
    active: boolean = true
) {
    return { name, role, active };
}

createUser("Alice");                    // default role & active
createUser("Bob", "admin");             // default active
createUser("Charlie", "mod", false);    // all specified
```

Important: In TypeScript, every function parameter must have its type explicitly defined. Unlike JavaScript, you cannot leave parameters untyped. This requirement ensures type safety throughout your codebase and enables the compiler to catch type-related errors early in development.

```typescript
// Every parameter needs a type annotation
function process(name: string, count: number, active: boolean) {
    // All parameters have explicit types
}

// This would cause an error in TypeScript:
// function process(name, count, active) { }
// Error: Parameter 'name' implicitly has an 'any' type

// Even callback parameters need types
function fetchData(callback: (data: string) => void) {
    callback("result");
}
```

Rest Parameters

Rest parameters accept any number of arguments as an array:

```
// Rest parameters with type annotation
function sum(...numbers: number[]): number {
    return numbers.reduce((total, n) => total + n, 0);
}

sum(1, 2);           // 3
sum(1, 2, 3, 4, 5);  // 15

// Rest with other parameters
function buildName(first: string, ...rest: string[]): string {
    return first + " " + rest.join(" ");
}

buildName("John", "Paul", "Smith");  // "John Paul Smith"

// Spread in function calls
const nums: number[] = [1, 2, 3];
sum(...nums);  // 6
```

Understanding Spread in Function Calls

The spread operator (`...`) allows you to expand an array into individual arguments when calling a function. This is the opposite of rest parameters - while rest parameters collect multiple arguments into an array, spread expands an array into separate arguments.

```
// Without spread - you'd have to pass each element manually
const numbers = [10, 20, 30];
sum(numbers[0], numbers[1], numbers[2]);  // 60 - tedious!

// With spread - the array is expanded into individual arguments
sum(...numbers);  // 60 - much cleaner!

// How it works:
// sum(...numbers) is equivalent to sum(10, 20, 30)

// Combining arrays with spread
const moreNumbers = [40, 50];
sum(...numbers, ...moreNumbers);  // 150

// Spread with Math functions
const values = [5, 10, 3, 8, 1];
Math.max(...values);  // 10
Math.min(...values);  // 1

// TypeScript ensures type safety with spread
const strings = ["a", "b", "c"];
sum(...strings);  // Error: Argument of type 'string' is not
                  // assignable to parameter of type 'number'
```

Tip: The spread operator is especially useful when working with arrays of unknown length, or when you want to pass array elements as individual function arguments without modifying the original function.

Function Overloading

Function overloading lets you define multiple signatures for one function:

```typescript
// Overload signatures
function format(value: string): string;
function format(value: number): string;
function format(value: Date): string;

// Implementation
function format(value: string | number | Date): string {
    if (typeof value === "string") {
        return value.toUpperCase();
    } else if (typeof value === "number") {
        return value.toFixed(2);
    } else {
        return value.toISOString();
    }
}

format("hello");           // "HELLO"
format(3.14159);           // "3.14"
format(new Date());        // "2024-01-15T...
```